Detecting persistent structural breaks by rebuilding the data generator, learning from its synthetic series, and TabPFN — team BreakPoint

Summary. Final Kaggle selections: **blendXFS** public **0.87791**, OOF 0.8741, and blendTFS public 0.87641, OOF 0.8724 (recipe: TRAINING_LATEST.md §6). Qualification submission: blendG2 (public 0.86386, OOF 0.8657); CPU-only reference blendD_r12 (public 0.86076), reproduced bit-for-bit by `predict.py` in 31 s on 2 cores. The core idea: **rebuild how the data was made, then learn from synthetic data**. We fitted a generator to the training series for realism (real-vs-fake classifier AUC 0.66 → 0.58, no labels), trained detectors only on its series (a public Chronos-2 model fine-tuned as a break classifier reaches AUC 0.845 on the real training set without ever seeing its labels), and fed them with our change statistics into TabPFN arms and a TabPFN stacker. Measured gains: residual-transform statistics (+0.008 OOF), TabPFN (+0.006), time reversal (+0.002), synthetic detectors and stacker (0.8684 → 0.8741, §3).

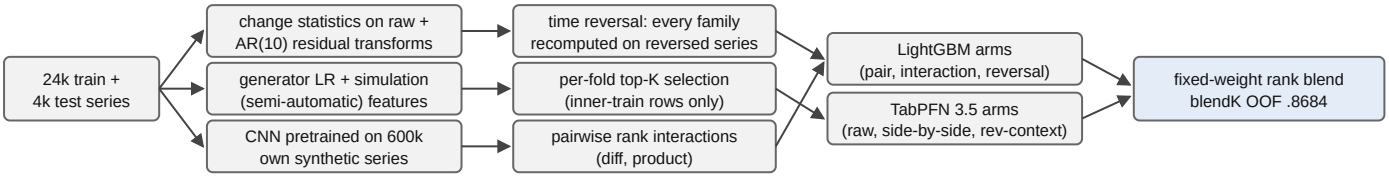


Figure 1. Core pipeline (blendK; the final blends add synthetic-trained detectors and a TabPFN stacker, §3). Label-dependent steps are fitted inside each training fold; the CNN and reversed features use no labels.

1. Task and validation design

Each series is a univariate sequence with a known boundary (period 0 / 1); the target is whether a lasting process intervention happened at the boundary (ROC-AUC). A change can be in level, variance, dependence or the noise distribution; in our calibrated generator about one in three level / variance breaks is only ~0.25 sd, invisible by eye. All numbers below are out-of-fold (OOF) over **StratifiedKFold(5, shuffle, seed 42) on whole series** (24,000 training series; no series is split). Everything learned from data is fitted inside the training part of a fold: early stopping uses an inner 10% split of the fold-train rows; feature selection for interaction features is redone in every fold on the inner-train rows only; pseudo-label and label-cleaning experiments were nested. A reversed copy always stays in the fold of its original series, so a validation series never enters training or a TabPFN context in either direction. The synthetic CNN is pretrained without any real label. Differences between variants are judged with a paired bootstrap of the OOF AUC (95% CI) and by per-fold agreement, never by the public leaderboard (public AUC on ~1,520 rows has a standard deviation of about 0.009, larger than most effects we test; OOF on 24,000 series about 0.002). Each change is first screened fast, then confirmed at full settings; quick gains usually halve.

2. Features (all per series, computed independently of labels)

- **Classical change statistics** (`features.py`, `harvest.py`): pre/post moments, quantiles, variance ratios, Levene/KS/Mood/AD/energy tests, CUSUM/Chow-type statistics, autocorrelation and spectral changes, boundary windows of several widths. Ideas adapted from public write-ups of the 2025 ADIA structural-break challenge (no code copied; see README).
- **Residual transforms** (`factory.py`): the same statistics on AR(10) prediction residuals and their absolute values fitted on the pre segment — the largest single gain of the project (OOF 0.833 → 0.841).
- **Generator-informed likelihood features**: Gaussian likelihood-ratio tests for level / spread / dependence / noise-type changes with a drift-aware null (`probe/joint_lr.py`, `probe/noise_lr.py`, `probe/gen_feats.py`, `probe/model_feats.py`).
- **Semi-automatic simulation features** (`probe/ku_gen.py`, `probe/sim_series_ku2.py`; team-generated synthetic data, allowed by the rules): a generator calibrated to the training data (WN/AR/SIN/RW/local-level families, t/GARCH noise, level/spread/dependence/shape breaks, nuisances) produces 30k labelled fakes per bag. LightGBM estimators trained on them predict the hidden generator parameters (family, break kind and sizes, noise type) of each real series: 31 columns, averaged over 6 bags.
- **Time-reversal augmentation**: a structural change at the boundary is still a change when the series is read backwards, so every series is reversed (boundary at $n-n_0$, same label), all feature families are recomputed on the reversed series, models are trained on originals + reversed copies and predict the mean of both (test-time augmentation). Single pair arm 0.8575 → 0.8599 OOF (+0.0025, CI [+0.0009, +0.0041]). Negation and cropping were tested the same way and did not hold up.
- **Synthetic-pretrained CNN features** (`probe/sim_tensor.py`, `probe/seq_pretrain.py`, `probe/seq_emb.py`): a dilated 1-D CNN (64 channels) is pretrained on 600k of our own synthetic series, aligned at the boundary (z-scored values, AR(10) residuals, period mask). It predicts break / no break plus the hidden generator parameters (break kind, family, break sizes, noise type, GARCH, decoy, trend) as auxiliary tasks. Without seeing any real label it reaches AUC 0.81 on the real training series (0.84 after continued pretraining on the realism-fitted generator, §3). Its 512-d embedding (PCA to 32) and the head outputs become 53 label-free features, computed for every series and for its reversed copy. Two extra pretraining epochs left zero-shot AUC unchanged (0.810) but raised the best TabPFN arm from 0.8658 to 0.8672.
- **Pairwise rank interactions** (`probe/confirm_pair.py`): in each fold, the top-K features by gain of a quick model fitted on the fold's inner-train rows are rank-transformed and all pairwise differences and products are added (K=60 gives 3,540 columns). This lifted the best single model from 0.8559 to 0.8575 OOF.

3. Models and the final blend

LightGBM arms (lr 0.02, 31 leaves, min_child 40, feature_fraction 0.5, bagging 0.8, early stopping 300 on the inner split, 3 seeds per fold). **TabPFN arms**: the public pretrained TabPFN 3.5 run on each fold's top-K features (K 40–100, whole fold-train as context, 8

estimators). The best TabPFN input is the raw values of the top-K features plus the synthetic-pretrained CNN features, for the series and for its time-reversed copy side by side (single-arm OOF 0.8672). The final submission is a fixed-weight rank blend of arm groups (each group = mean of ranks):

blendK = rank(2·pair + 1·interaction + 2·reversal-pair + 6·rev-context TabPFN + 9·CNN-feature TabPFN + 3·rev-context CNN TabPFN + 9·raw TabPFN + 3·rev-context raw TabPFN + 9·raw side-by-side TabPFN + 24·raw side-by-side TabPFN with longer-pretrained CNN). The CPU fallback blendD uses only the first two groups.

The 11 pair arms differ in K (40–80), the seed of the per-fold feature selection and the semi-automatic feature bag; the interaction arm takes the top-162 pool features, builds ~91k pairwise operations (product, sum, difference, ratios, $a \cdot b^2$, squares) inside each fold, and retrains on the 800 highest-gain columns. The LightGBM weights (2/1/2) were chosen by cross-validation over a small grid (picked on 4 folds, scored on the 5th). Each later TabPFN group was added with one weight from {3, 6, 9, 24}, kept only if the OOF gain had a bootstrap CI above zero and held in all 5 folds. These weights were chosen on the full OOF, so the last gains (+0.0003 to +0.0007 each) may be slightly optimistic. No weight was ever tuned on the public leaderboard.

Model / blend	OOF AUC	Public AUC
LightGBM, 242 classical features (baseline pipeline)	0.8327	–
+ residual transforms, generator LR features (combo10gnj)	0.8539	0.85415
+ semi-automatic simulation features (single LGB, bag6w)	0.8559	–
blendD / blendD_r12 (CPU fallback): 11 pair arms ×2 + interaction ×1	0.8606	0.86076
blendG2 (qualification): + time-reversal pair arms + reversed-context TabPFN	0.8657	0.86386
+ TabPFN with synthetic-pretrained CNN features (blendH)	0.8669	–
blendK: + side-by-side TabPFN on longer-pretrained CNN features	**0.8684**	**0.86971**
blendNFS: + nuisance-pretrained CNN, Chronos-2, TabPFN stacker	0.8706	0.87416
blendOFS: + TimesFM-2.5 group	0.8710	0.87356
blendRFS: + Chronos-2 fine-tuned on synthetic series as a break classifier	0.8719	0.87550
blendTFS (selected): + classifier retrained on the realism-fitted generator genR10	0.8724	0.87641
blendXFS (selected): + realistic fakes in the estimators, segment noise classifier	**0.8741**	**0.87791**

Each step is a paired-bootstrap improvement that is higher in all 5 folds: blendK +0.0007 [+0.0003, +0.0010] vs blendJ. Stacked rows (NFS–XFS) are outer-CV OOF of the TabPFN stacker (seeds 11 / 13 agree within 0.0001).

From blendK to blendXFS: learning from a rebuilt generator. (1) A TabPFN **stacker** on the logits of all arms plus label-free context replaces the fixed weights (+0.0004 to +0.0005 over the linear blend at each step). (2) **Generator fitted for realism**: we changed one generator setting at a time and kept it only if a real-vs-fake classifier (LightGBM on 34 segment statistics, mean of 3 fake sets) found the fakes harder to spot: genT 0.662 → genR10 0.580 (bimodal and GARCH noise only in white noise, lighter t tails, spikes, smaller trends). (3) **Chronos-2 as a break classifier** (public 120M forecaster, encoder of 512 points on each side of the boundary, two variates: z-series and AR residual), trained only on synthetic series: zero-shot AUC (forward view) 0.8408 → 0.8438 after retraining on genR10, 0.8455 with the reversed view averaged in; its embeddings feed new TabPFN groups. (4) **Segment noise classifier**: an oracle test (LightGBM on 90k fakes, tested on 30k held-out fakes, base 0.8154; the true value of one hidden setting added as a feature) showed that knowing the pre-segment noise type and tail is worth +0.0070 AUC, more than all other settings together (GARCH +0.0023, trend +0.0016, family +0.0013, AR +0.0011, sine +0.0011), so a LightGBM trained on genR10 fakes predicts noise type, tail weight (1/df) and volatility clustering (GARCH) from AR(5)-whitened residuals of each segment (type accuracy 0.82 on fakes), applied before and after the boundary. It catches breaks a variance test misses (same spread, heavier tails). It helped at every level: best LightGBM 0.8567 → 0.8588 (+0.0021, CI [+0.0011, +0.0032]), best TabPFN arm 0.8716 → 0.8729, stacker 0.8733 → 0.8741, public 0.87641 → 0.87791. Negative results of this phase: CHANGELOG_FINAL.md.

Ablation of blendK (groups removed, other weights unchanged)	OOF AUC	Δ [95% CI]	folds down
blendK (all 10 groups)	0.8684	–	–
− all TabPFN groups (LightGBM only)	0.8627	−0.0057 [−0.0067, −0.0044]	5/5
− all groups that use time reversal	0.8665	−0.0019 [−0.0026, −0.0013]	5/5
− groups with synthetic-pretrained CNN features	0.8677	−0.0007 [−0.0009, −0.0004]	5/5
− all LightGBM groups (TabPFN only)	0.8684	+0.0001 [0.0000, +0.0002]	1/5

LightGBM arms add no AUC on top of TabPFN here, but they carry the CPU route and choose the top-K features of every TabPFN arm.

4. Error and robustness analysis

- **By series family** (estimated by the semi-automatic models; final blendXFS OOF): sinusoid 0.911, AR(2) 0.885, white noise 0.874, AR(1) 0.869, **random walk 0.709** (blendK 0.710, CPU blendD 0.702). Random walks stay the weak spot: an oracle on simulated random walks that knows the true parameters reaches only ~0.76, and they are equally hard on synthetic data, so most of this gap looks intrinsic. By series length (quintiles, shortest to longest): 0.821, 0.847, 0.880, 0.894, 0.911.
- **Train/test shift**: adversarial validation separates train from test at AUC 0.59. The driver is a group of test series whose low-order floating-point bits look rescaled (17.9% of test vs 1.7% of train); these series are almost white noise. blendK is less certain on them (mean |rank − 0.5| 0.19 vs 0.26) and only slightly shifted (mean rank 0.53 vs 0.49). Importance re-weighting towards the test distribution hurt OOF (−0.0047), and fold-clean pseudo-labelling gave only +0.0009 for one arm, so neither is in the final blend.

- **Leaderboard noise**: AUC of blendK on 4,000 random 1,520-row subsets of the training OOF (the public-test size) has standard deviation 0.0089 (95% range 0.850–0.885); on 2,480 rows (the private-test size) it is 0.0067. Our public 0.86971 is in line with the OOF 0.8684. Public gaps between teams below about 0.01 are therefore not reliable, which is why we choose models by OOF and never tune on the public score.
- **Ceiling**: on our simulator, a likelihood-ratio oracle that knows the true parameters reaches ~0.91. The same oracle with parameters estimated from each series reaches ~0.86–0.87. blendK (0.8684) is already at this plug-in level, so further gains need better estimates of spread, dependence and shape changes on short segments, not more model capacity.
- **What did not help** (each tested with paired-bootstrap CIs; Δ = change of the best blend at that time):

Idea	Result	Idea	Result
XGBoost / CatBoost / ExtraTrees / MLP / kNN arms	Δ ≤ 0	Negation and cropping augmentation	Δ 0 / −0.004
Raw-sequence CNNs, MiniRocket kernels	OOF 0.75–0.83, Δ ≤ 0	Fine-tuned CNN (real labels)	OOF 0.818, hurts blend
Label-noise down-weighting	+0.0011, n.s.	TabICL instead of TabPFN	OOF 0.8567, Δ −0.0002
Pseudo-labels (LightGBM / TabPFN context)	+0.0009 / 0	Second simulator, self-supervised detector	corr 0.99, Δ 0
Importance weighting to test shift	−0.0047	Chronos forecast-error features	hurts TabPFN
Logistic / LightGBM stacking	−0.0020 / −0.0007	Distilling TabPFN into LightGBM	student 0.8572, Δ 0
Dozens of extra feature families (MOSUM, ordinal patterns, MMD, BDS/Hurst/ADF, change-point scans): each ≤ +0.0001.			

5. Limitations

- The synthetic generator is a reverse-engineered approximation: real-vs-fake AUC 0.58 on segment statistics but 0.65 on break-specific statistics (synthetic breaks are slightly smaller than real ones, mostly in noise shape); every model trained on fakes inherits its biases. Fine-tuned Chronos steps vary by about ±0.004 zero-shot AUC between fake chunks.
- Many variants were compared on the same OOF folds; reported gains are paired-bootstrap CIs but not corrected for multiple comparisons, and quick-screen gains typically halved at production settings.
- **Resources.** Our best models need a GPU; blendXFS stacks 296 arms, but the best single TabPFN arm (0.8729) is within 0.0012 of the stacker, so most arms are redundant. TabPFN did not finish predicting 100 test rows in 7 minutes on 2 CPU cores, and distilling it into LightGBM lost the gain. The CPU-only route reaches OOF 0.8641 (LightGBM + reversal + CNN-feature pair arms), 0.0043 below blendK. We therefore ask that the final audit allow **one consumer GPU (≥16 GB)**:
- Full training (features, 6 synthetic bags, interaction arm) takes several hours on a multi-core HPC node; the audit route uses frozen checkpoints plus the documented commands (TRAINING.md).

6. Reproducibility, hardware, sources and AI use

Submission	Route	How to reproduce	Cost
blendD_r12 (CPU fallback)	LightGBM only	`predict.py` + frozen checkpoints in `ckpt/`; output equal to the Kaggle file bit for bit	31 s, 1 GB, 2 cores
blendG2 (qualification)	+ TabPFN	TRAINING_GPU.md + `make_blendG2.py`; reproduces the Kaggle file exactly	minutes on one GPU
blendK … blendXFS / blendTFS (final)	+ CNN, Chronos-2, stacker	TRAINING_LATEST.md §1–6, `ckpt/seq/`, `manifests/`; fine-tuned Chronos-2 checkpoints (~486 MB) by SHA-256, on request	one GPU

Rounding rule: every arm vector is rounded to 12 decimals before ranking, so ties do not depend on floating-point noise. **Training hardware**: KU Madar HPC CPU nodes (64–96 cores, Slurm), a local 32-core workstation; two local RTX 4090 GPUs for the TabPFN arms and for CNN pretraining (one GPU is enough). Python 3.12/3.13, LightGBM 4.6.0, scikit-learn 1.7, NumPy 1.26, pandas 2.3, SciPy 1.16 (requirements.txt). Randomness: every split, model and generator seed is fixed in code. **Data**: only the supplied competition data plus team-generated synthetic series (generator code included); no external datasets. **Sources**: public ADIA 2025 write-ups (ideas only); Page (1954) CUSUM; Andrews (1993) tests for unknown change points; Dempster et al. (2020) ROCKET; Hollmann et al. (2025, *Nature*) TabPFN, version 3.5 public pretrained weights; Bai et al. (2018) temporal convolutional networks (dilated CNN design); Chronos (Ansari et al., 2024) / Chronos-2 and TimesFM-2.5 (Das et al., 2024) public forecasters; time-reversal augmentation follows the general data-augmentation literature for time series (Wen et al., 2021). **AI use**: an AI coding assistant (Anthropic Claude, via Claude Code) helped write and run analysis code, design experiments and draft this report under continuous human review; the assistant executed code on our own machines; its context contained summary statistics, scores and occasionally a few printed values of individual series while debugging, but no competition files, label files or prediction files were uploaded to any third-party AI service (data stayed on the local workstation and the university HPC). Details in AI_DISCLOSURE.md.

7. What we learned

- **Representation beat model choice**: every large gain was new information; swapping learners on the same features never helped.
- **Use the task's structure**: a lasting change is still a change when read backwards (time reversal).
- **Rebuild the data, then learn from it**: fitting the generator for realism and training detectors only on its series gave the last gains without using real labels; an oracle on fakes told us which error was still fixable.
- **Measure noise first**: paired bootstrap CIs plus a 5-of-5-folds rule filtered out dozens of false leads (§4).